

作业 8 异常与外设中断

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博

学号 2351520

Part 1、异常，备考期末

1、（Ubuntu 系统）输入除数 0，描述异常响应过程

```
#include <unistd.h>
#include <stdio.h>
int
main(void)
{
    int a,b,c;

    for ( ; ; )
    {
        printf("输入被除数\n");
        scanf("%d", &a);

        printf("输入除数\n");
        scanf("%d", &b);

        c=a/b;

        printf("%d / %d = %d\n",a,b,c);
    }
}
```

当程序执行到除法指令且除数为 0 时，并未定义信号处理函数（Signal Handler），系统响应过程如下：

1. CPU 检测与异常抛出：CPU 执行除法指令（如 `idiv`），运算单元（ALU）检测到除数为 0。CPU 硬件立即停止当前指令执行，并抛出 0 号异常（Divide Error）。
2. 保护现场：CPU 将当前程序计数器（EIP/RIP，指向发生错误的除法指令）、状态寄存器（EFLAGS）等压入核心栈。
3. 陷入内核：CPU 从用户态切换到核心态，查中断向量表（IDT），跳转到操作系统内核预设的 0 号异常处理程序。
4. 内核发送信号：0 号异常处理程序识别出是除零错误，向当前运行的进程发送 SIGFPE 信号（8 号信号）。
5. 默认处理：由于应用程序代码中没有定义针对 SIGFPE 的信号处理函数，操作系统执行该信号的默认动作：终止（Kill）该进程。
6. 结果：进程异常退出，终端输出 "Floating point exception (core dumped)"。

```
ubuntu@SYUDY002:~/vscode-test$ ./1
输入被除数
10
输入除数
0
浮点异常（核心已转储）
```

2、(Ubuntu 系统) 输入除数 0，描述异常响应过程（突出用户栈、核心栈的变化）

应用程序设置信号处理函数后，除0异常杀不死进程



```
noException.c x divide0.c x ctrlc2.c x exception.c
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

static void sig_dzero(int);

int
main(void)
{
    int a,b;
    int c;

    if (signal(SIGFPE, sig_dzero) == SIG_ERR)
        printf("can't catch divide by zero error!");

    for ( ; ; )
    {
        printf("输入被除数\n");
        scanf("%d", &a);

        printf("输入除数\n");
        scanf("%d", &b);

        c=a/b;
        printf("%d / %d = %d\n",a,b,c);
    }
}
```

```
static void
sig_dzero(int signo) /* argument is signal number */
{
    printf("Divide by zero!\n");
}
```

操作系统(拔尖班 / 国豪 2025)

同济大学计算机系 邓蓉

12

当程序中设置了 `signal(SIGFPE, sig_dzero)` 后，流程发生了变化，导致了死循环：

1. 异常触发与保存现场（核心栈变化）：CPU 抛出 0 号异常。核心栈中压入用户态的 SS、ESP（用户栈顶指针）、EFLAGS、CS、EIP（指向发生错误的除法指令）。
2. 内核处理与信号标记：内核执行 0 号异常处理程序，将 SIGFPE 信号发送给进程（设置进程控制块中的 `u_signal[8]`）。
3. 准备返回用户态与堆栈篡改：在异常处理结束准备返回用户态前，内核检测到该进程有挂起的 SIGFPE 信号且定义了自定义处理函数 `sig_dzero`。
4. 核心栈修改：内核将核心栈中保存的 EIP 修改为信号处理函数 `sig_dzero` 的入口地址。
5. 用户栈修改（扩展）：内核修改核心栈中保存的 ESP 值，使其向低地址移动（扩展用户栈）。在用户栈的新栈帧中，内核压入两个关键数据：返回地址（Old EIP）：即原本发生异常的那条除法指令的地址。信号参数：信号编号（8）。这样做的目的是保存断点，并为信号处理函数传递参数。
6. 执行信号处理函数：执行 IRET 指令，CPU 返回用户态。此时 EIP 指向 `sig_dzero`，程序开始执行打印 "Divide by zero!"。
7. 死循环：信号处理函数 `sig_dzero` 执行完毕，执行 `ret` 指令。CPU 从用户栈弹出之前内核压入的“返回地址”。这个地址正是那条会导致除零错误的除法指令。CPU 再次执行除法指令 -> 再次触发 0 号异常 -> 再次进入信号处理 -> 再次返回执行除法。进程陷入无限循环，不断打印错误信息。

```
ubuntu@SYUDY002:~/vscode-test$ ./2
输入被除数
10
输入除数
0
```

```
Divide by zero!
Divide by zero!
Divide by zero!^C
```

3、(Ubuntu 系统) debug 第二小问展示的这个程序

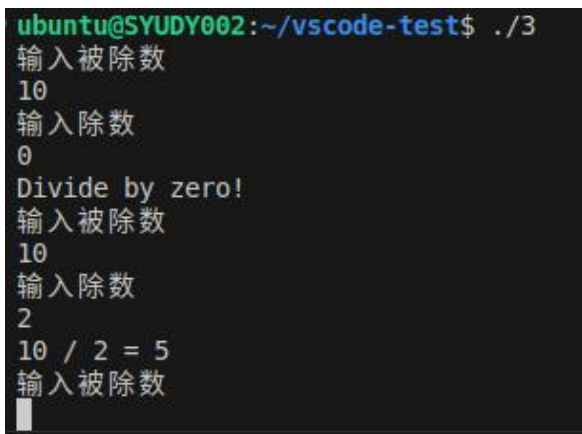
简单的信号捕获后返回会导致重新执行故障指令（除以 0），从而死循环。所以，需要使用远程跳转技术，跳过故障指令，回到程序的安全点，比如循环的开头。

以下是修改后的程序：

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>
#include <setjmp.h>

static sigjmp_buf fpe_env; // 定义一个全局缓冲区，用来保存 CPU 的“现场”
static void sig_dzero(int signo)
{
    printf("Divide by zero!\n");
    siglongjmp(fpe_env, 0); // 执行远跳转，回到 main 函数中 sigsetjmp 调用的位置
}

int main(void)
{
    int a, b;
    int c;
    if (signal(SIGFPE, sig_dzero) == SIG_ERR)
        printf("can't catch divide by zero error!\n");
    for (;;)
    {
        sigsetjmp(fpe_env, 1); // 设置一个跳转点，保存 Signal Mask（信号屏蔽字）
        printf("输入被除数\n");
        if (scanf("%d", &a) != 1) break;
        printf("输入除数\n");
        if (scanf("%d", &b) != 1) break;
        c = a / b;
        printf("%d / %d = %d\n", a, b, c);
    }
    return 0;
}
```



```
ubuntu@SYUDY002:~/vscode-test$ ./3
输入被除数
10
输入除数
0
Divide by zero!
输入被除数
10
输入除数
2
10 / 2 = 5
输入被除数
█
```

Part 2、外设中断

- 1、情景分析，系统并发 copy 进程 和 matrix 进程（4096*4096 矩阵乘法）。请画甘特图、描述这两个进程轮流使用 CPU 的情景（不考虑 write 系统调用）。



四、读操作：copy 进程执行 read 系统调用读磁盘文件 oldFile

CPU t0 t1 t2 t3 t4

磁盘 _____

```
int main(int argc, char* argv[])
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1], O_RDONLY);
    int newFile = open(argv[2], O_WRONLY|O_CREAT|S_IRUSR|S_IWUSR);

    char c;
L: while( read(oldFile, &c, 1) == 1 )
    write(newFile, &c, 1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
copy oldFile newFile
```

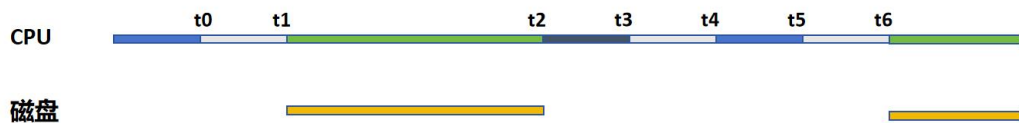
操作系统(拔尖班 / 国豪 2025) 同济大学计算机系 邓蓉 26

- t0时刻，copy 进程执行**read 系统调用上半段**：时刻t1发磁盘读命令，入睡放弃CPU
- [t1,t2]，磁盘和DMA控制器为copy进程服务，将磁盘上的文件数据读入核心态内存（一个缓存块）
- t2时刻，IO完成。被中断的现运行进程PA执行**磁盘中断处理程序**唤醒copy 进程。。。
- t3时刻，copy 进程得到CPU**执行read 系统调用下半段**，送数据进用户空间c变量。
- t4时刻，copy 进程返回用户态执行应用程序。

Copy 进程：I/O 密集型。主要行为是 read()（读磁盘）-> 阻塞 -> write()。

Matrix 进程：CPU 密集型。4096*4096 矩阵乘法，持续占用 CPU 计算。

甘特图：



- t0 时刻：copy 进程正在运行，调用 read 系统调用。CPU 执行内核代码（系统调用上半部），向磁盘控制器发送“读取文件”的命令。
- t1 时刻：read 系统调用的上半部执行完毕。由于磁盘读取很慢，copy 进程无法继续执行，进入阻塞 (Sleep) 状态，主动放弃 CPU。操作系统调度器介入，发现 copy 睡了，于是选择就绪队列中的 matrix 进程上台。此时，磁盘在为 copy 读取数据，而 CPU 在为 matrix 进行计算。
[t1 ~ t2]: matrix 进程占用 CPU 进行矩阵运算；磁盘控制器 (DMA) 持续将数据从磁盘搬运到内存中的内核缓冲区。
- t2 时刻：磁盘 I/O 操作完成。磁盘控制器向 CPU 发送一个中断信号。CPU 必须响应中断：它打断当前正在运行的 matrix 进程，保存 matrix 的上下文（压栈），跳转到中断向量表对应的入口。
[t2 ~ t3]: CPU 执行磁盘中断处理程序。内核确认数据读取完毕，将 copy 进程的状态从“阻塞”改为“就绪 (Ready)”。因为 I/O 密集型进程 (copy) 刚刚完成 I/O，为了让磁盘尽快再次工作，操作系统通常会提升其优先级。
- t3 时刻：中断处理结束。调度器判断 copy 进程优先级高于 matrix 进程。CPU 切换回 copy 进程（恢复其内核栈上下文）。
[t3 ~ t4]: copy 进程继续执行 read 系统调用的下半部。CPU 将数据从内核缓冲区 copy 到用户空间的变量 c 中。

- t4 时刻：系统调用完全结束，copy 进程返回用户态，继续执行下一条指令。
- t5 时刻：copy 进程调用 read 系统调用。CPU 执行内核代码（系统调用上半部），向磁盘控制器发送“读取文件”的命令。
- t6 时刻：read 系统调用的上半部执行完毕。copy 进程进入阻塞（Sleep）状态，主动放弃 CPU。操作系统调度器介入，选择就绪队列中的 matrix 进程上台。此时，磁盘在为 copy 读取数据，而 CPU 在为 matrix 进行计算。
- 此后上述过程一直循环，两个进程轮流使用 CPU.....

归纳总结：计算机系统怎样执行 IO 操作。

计算机执行 I/O 操作是一个软硬件协同的过程，主要步骤如下：

1. 应用程序请求：用户进程调用系统调用（如 read）。
2. 内核指令：操作系统内核接收请求，通过设备驱动程序向外设控制器发送 I/O 命令。
3. 进程阻塞：由于外设速度慢，CPU 不会忙等待。现运行进程（发起 I/O 的进程）被标记为阻塞/睡眠状态，让出 CPU。
4. 调度：操作系统调度其他就绪进程（如 matrix）使用 CPU。
5. 外设工作：外设控制器控制硬件设备完成实际的读写操作。
6. 中断通知：当 I/O 完成后，外设控制器通过中断控制器向 CPU 发送中断请求信号。
7. 中断响应：CPU 暂停当前任务，保护现场，执行中断处理程序。
8. 数据就位与唤醒：中断处理程序确认数据传输完成（通常通过 DMA 放入内存），唤醒之前等待 I/O 的那个进程（将其设为就绪态）。
9. 恢复执行：该进程下次获得 CPU 时，从系统调用返回，拿到 I/O 数据继续运行。

延伸：scanf 读键盘，进程执行的也是 read 系统调用：

read(0,***,***)读标准输入文件

用户敲键盘输入字符。所以，读键盘，CPU 是不需要向外设发命令的，read 系统调用上半段，进程直接 Sleep 入睡。

scanf 本质也是调用 read(0, ...)。

当用户未输入时，进程睡眠。

用户敲击键盘产生键盘中断。键盘中断处理程序读取扫描码，唤醒等待 scanf 的进程。

所以，读键盘时 CPU 不需要向键盘发“读命令”，而是被动等待键盘的输入中断。

Part 3、（思考题，总评分加 5）Unix V6++ 系统，复现上述程序。